

Collecting Czech intraday energy data, reliably, so anyone can add a new source

A plain-language explanation of what we built, how it works, and how we got there. 19 September 2026.

In one minute. Electricity is traded and consumed in 15-minute blocks. Public websites publish prices and grid load for each block, but each site does it differently. We designed a system that downloads that data every few minutes, keeps the original files forever, converts everything into one clean table, and keeps working when a website, a server or a database fails. The part we care about most: a new person, or an AI assistant, can add a new data source by filling in a short "recipe", and automatic checks stop the common mistakes before a human even looks at it.

What this document covers

- **1. The task** in plain words, and what data we chose.
- **2. How the data flows** from a website to a clean table (diagram).
- **3. How it keeps working when things break** (diagram).
- **4. How someone new adds a data source** (diagram).
- **5. The guardrails:** what an automatic check stops, and why (diagram).
- **6. Where it runs and how updates are made safe** (diagram).
- **7. How we worked**, including an independent review that found real gaps (diagram).
- **8. Where we are now** and what comes next. Honest status.
- **9. Five words we could not avoid**, explained.

Three ideas that shape everything

Keep the original. Every file we download is stored exactly as received, in two places. If we ever misread a file, we fix the reader and re-run from the archive. Nothing is lost.

The recipe is data, not code. Adding a source means describing it (where, how often, what the columns mean) in a short file. The platform does the fetching and the converting. Newcomers cannot break the core because they never touch it.

Make the wrong thing fail. Rules that live only in a document get ignored. Every rule we care about has an automatic check, and a test that proves the check actually rejects a bad change.

1. The task, in plain words

The assignment asked for a system that **collects public intraday energy data**, keeps running even when parts of it fail ("high availability"), can be **set up from code** on a fresh environment, and, most importantly, is built so that **someone else can add a new data source easily**, whether that someone is a junior engineer or an AI coding assistant, without introducing sloppy engineering. The brief was deliberately vague about which data. Finding that out was part of the job.

What data we chose, and why

Source	What it gives us	Why it is a good first target
OTE , the Czech electricity market operator (web service)	For every 15-minute block of the day: the average traded price and the traded volume on the continuous intraday market.	Public, no login, documented by OTE itself. The core of "intraday energy data".
OTE , same market, daily spreadsheet	The same blocks plus extra detail: buy and sell volumes, lowest, highest and last price.	Richer numbers; also a good test that two feeds of the same thing agree.
ČEPS , the Czech grid operator (web service)	Electricity consumption ("load") of the whole country, per 15 minutes.	Physical reality to compare against market prices. Public and documented.
OTE day-ahead market (web service)	Yesterday's auction prices for today.	Not built by us directly: it is the demo of "someone else adds a source".

What we deliberately left out

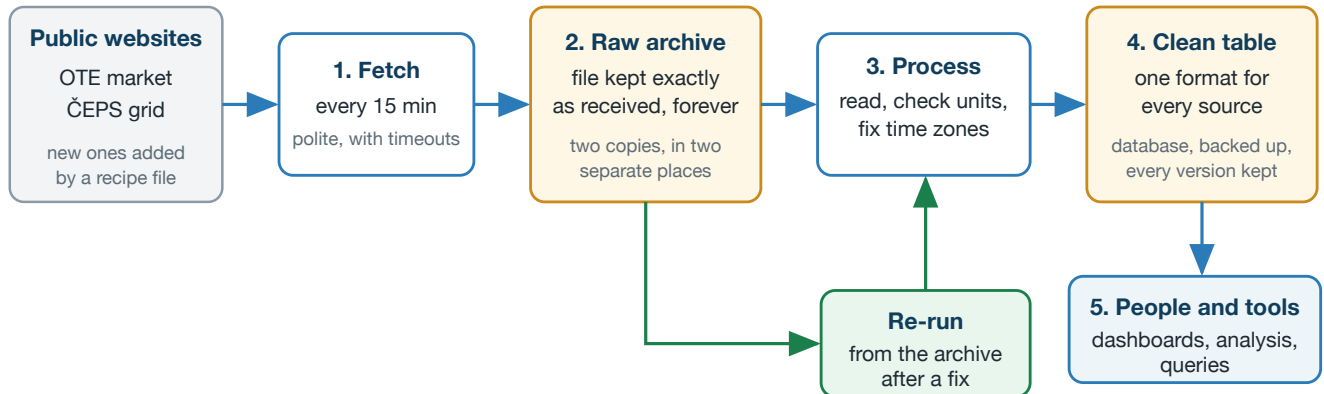
- **Exchanges that require a paid licence** (for example EPEX). The system refuses any source whose licence field is missing or marked "restricted".
- **Scraping web pages that forbid it**. ČEPS's website asks robots not to crawl it, so we use only its official data service.
- **Anything behind a login, captcha or rate limit** we would have to work around.
- **Redistributing the downloaded files** until the publishers confirm it is allowed. Our test files are look-alikes with the same shape, not copies.

A few things about this data that are easy to get wrong

- **Clock changes**. Twice a year a day has 23 or 25 hours, so 92 or 100 blocks instead of 96. The spreadsheet even repeats the label "02:00" in autumn. We count blocks from the calendar, never from the label.
- **Decimal commas**. Czech pages write 170,13 where a computer expects 170.13. We convert explicitly and never guess.
- **Empty is not zero**. A blank price means "no trade" or "not published yet", never 0 EUR.
- **Late corrections**. Publishers revise numbers. We keep every version and mark which one is current, instead of overwriting.

2. How the data flows

Five steps, always in this order. The key rule: **save the original before doing anything clever with it.**



Step 2 is what makes everything else safe: if step 3 has a bug, we fix the reader and re-run. The original files never change.

Step 4 never overwrites: a corrected number is added as a new version, and the old one stays visible for audit.

Diagram 1. From a public website to a clean, queryable table. Green: the repair path that the archive makes possible.

What "one format for every source" means

Every row in the clean table answers the same questions: which 15-minute block (in universal time, with the local date and block number kept alongside), which source, which measurement, which unit, when the publisher said it was valid, when we fetched it, and which version of our reader produced it. Because every source lands in this shape, a dashboard built for OTE prices works unchanged for any source added later.

3. How it keeps working when things break

"High availability" here does not mean a big cluster that never goes down. It means: **no single failure loses data, and no manual step is needed to recover**. Each 15-minute run is a small job with checkpoints. Whatever fails, the next run knows where to pick up.

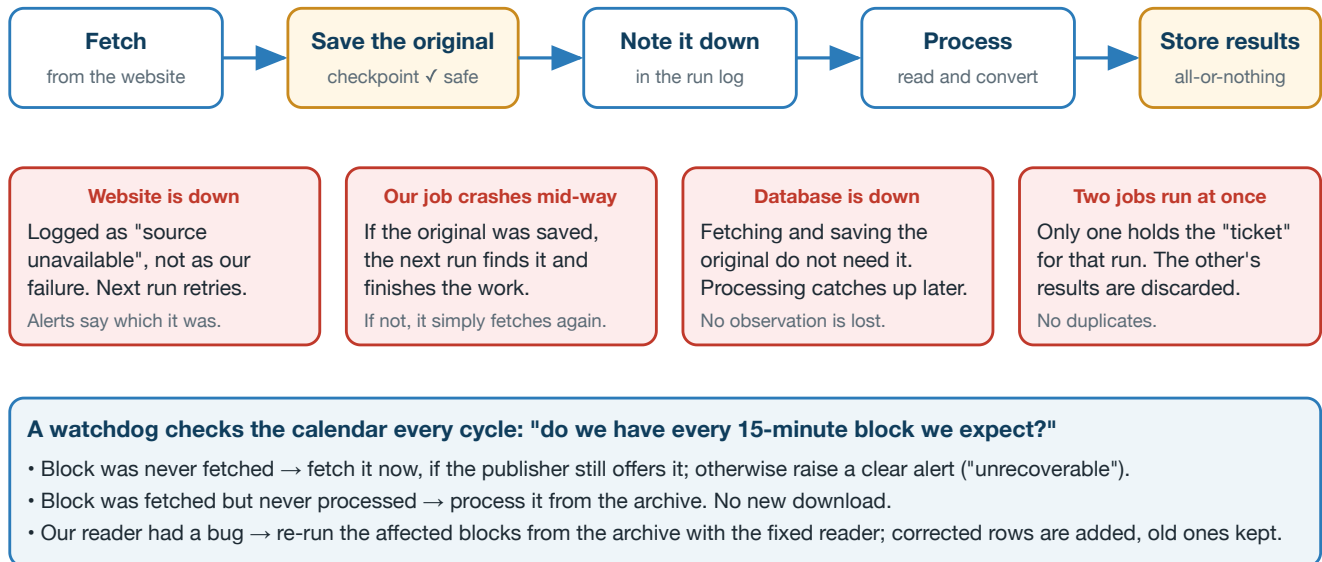


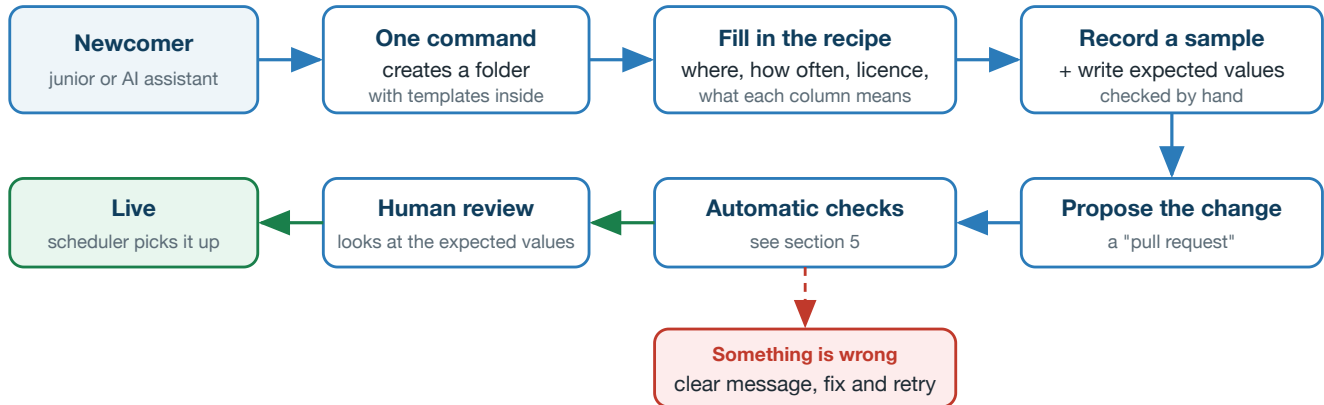
Diagram 2. Checkpoints in every run, and what happens at each kind of failure. Nothing here needs a person at 3 a.m.

How we know it works, rather than hope

- **Freshness is the measure.** The dashboard does not show "is the server up"; it shows "how old is the newest data for each source, compared with when the publisher should have released it". That is the number an operator cares about.
- **Drills, on a schedule.** A restore from backup and a rollback of a bad release are run automatically and checked. A backup that has never been restored is a guess.
- **Each failure above becomes a test** that is run on every code change, with the network and database simulated.

4. How someone new adds a data source

This is the heart of the assignment. The newcomer, human or AI, never writes downloading or conversion code. They fill in a **recipe** (a short text file), record a **sample**, and write down the **expected values** they checked by hand. Everything else is the platform's job.



If the source is entirely new to the platform (a kind of data or a website nobody approved yet):



Diagram 3. Top: adding a source the platform already approved. Bottom: what happens when the source is genuinely new. Guessing a unit to "make it pass" is the failure we designed against.

Why the newcomer cannot break the core

- Their change may only touch **their own folder**. Anything else is rejected automatically.
- Their folder may contain only **known kinds of files**: the recipe, a sample, expected values, and, rarely, a small reader for unusual file layouts. That reader receives data already opened by the platform, so it needs no network and no outside libraries.
- Approved units, measurement names and websites live in a **central list** that only maintainers can change. A recipe can pick from the list; it cannot invent.

5. The guardrails: what the automatic checks stop

We started from a list of the mistakes juniors and AI assistants typically make, and gave each one a **gate**. A gate is only real if there is a **test that proves it rejects the bad change**. Every row below has one.

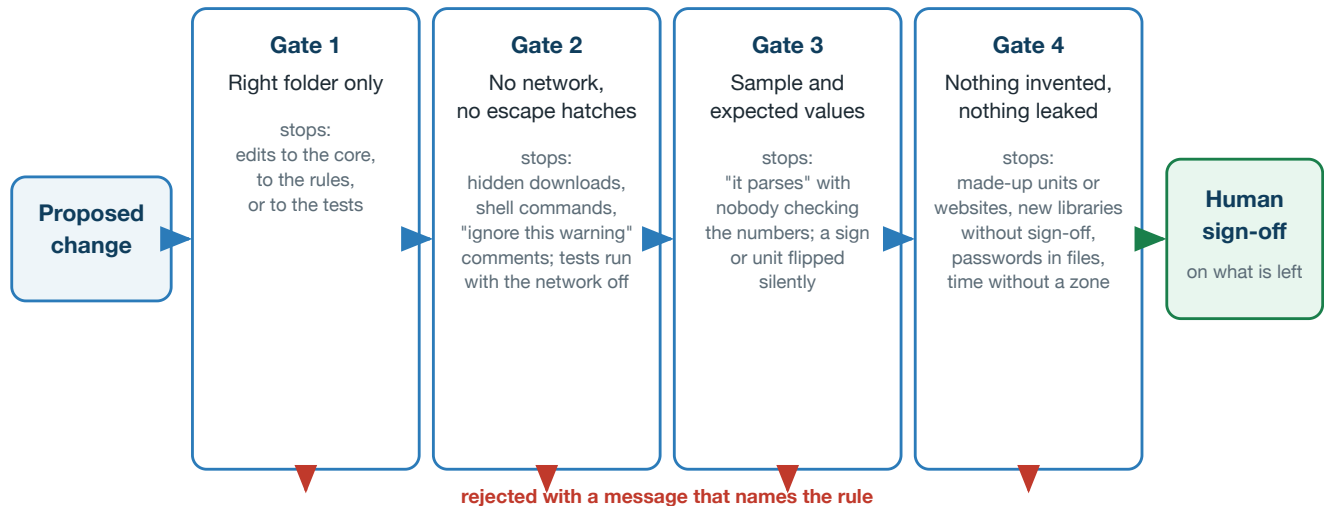


Diagram 4. A change must pass every gate before a person spends time on it. The person then reviews the one thing a machine cannot: whether the expected values are right.

Two rules about the rules

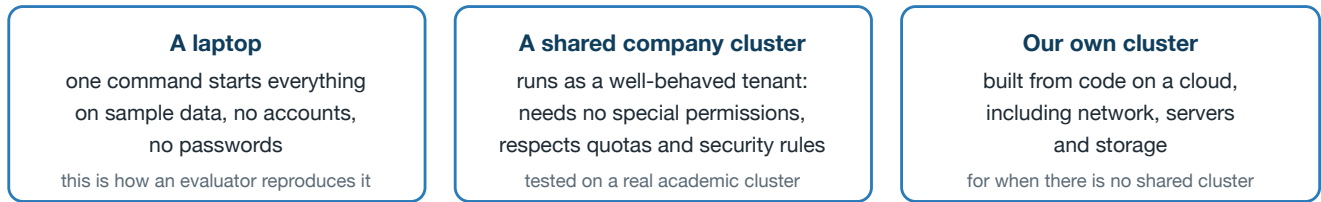
- **A gate can never be loosened to make a change pass.** The files that define the gates are owned by maintainers, and a newcomer's change cannot touch them (that is Gate 1 protecting the others).
- **Gates are guardrails, not a prison.** They catch accidents and make deliberate bypasses visible in review. For AI assistants working unattended, there is a separate locked-down mode with no shell and no network except the code repository. We say "guardrail" and "boundary" deliberately, and never confuse the two.

What the AI assistant is, and is not, allowed to do

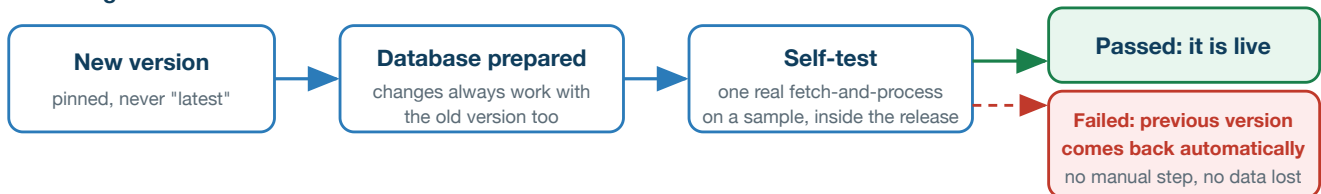
Level	Allowed
Read	Documentation, existing recipes, samples, logs.
Generate	A new recipe folder: recipe, sample, expected values, small reader, notes.
Propose	Open a pull request for a human to review.
Never	Merge its own change, see production passwords, change infrastructure, write to the production database, delete archives.

6. Where it runs, and how updates are made safe

Everything needed to run the system is written as code, so a fresh environment can be built from scratch and rebuilt identically. The same package runs in three places, differing only in a settings file.



Releasing a new version



Rolling back the code does not un-write bad numbers. That repair is the green "re-run from the archive" path in Diagram 1, which is why every row in the clean table remembers which version of the reader produced it.

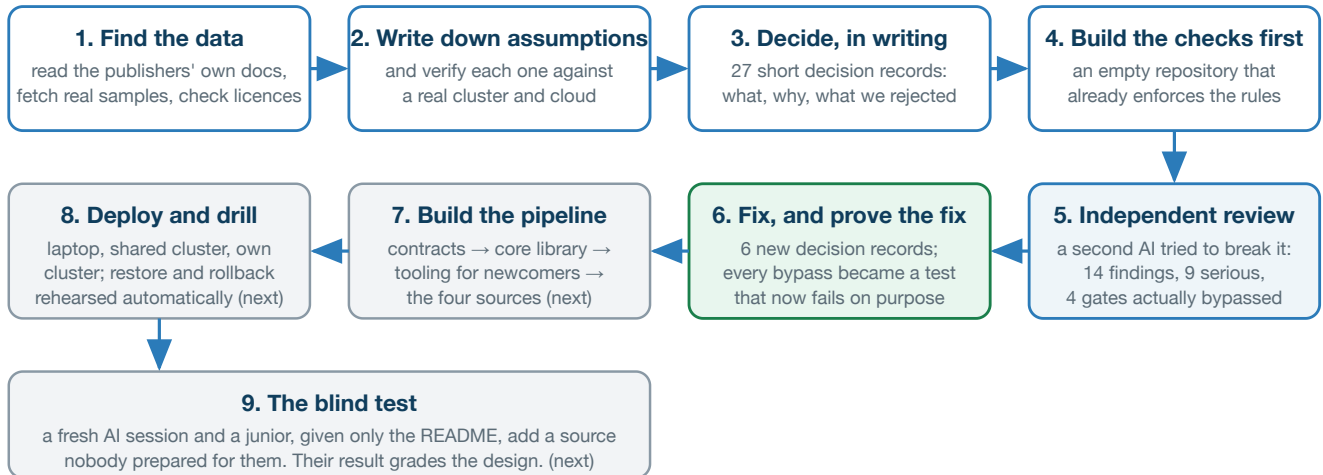
Diagram 5. Three places, one package. Below: the release sequence, with the self-test inside the release so that failure rolls back by itself.

Other protections worth knowing about

- **Backups in two places.** The archive is copied to a second, independently run storage service. The database has point-in-time backups. Both are restored on a schedule to prove they work.
- **Limited internet access.** Only the fetching jobs may reach the internet, only over encrypted connections, only to approved websites, and never to internal or private addresses. Redirects are checked hop by hop.
- **Nothing runs as an administrator.** Every part runs under the strictest security profile the cluster offers, with declared limits on memory and CPU.

7. How we worked

The assignment expected AI assistants to do much of the work and asked how we would lead and constrain them. Our answer was to **write the rules down first, get them independently reviewed, and make each rule mechanical** before writing the data pipeline itself.



Blue: done. Green: done this week. Grey: planned, in order. The loop from 4 to 6 is the part we would repeat before every phase.

Diagram 6. Rules first, review second, code third. The independent review is a step, not an afterthought.

How we lead the AI assistants day to day

- **One short instruction file** at the top of the repository states the hard rules and what the assistant may and may not do. It is identical for every assistant.
- **Plan before code.** Each work session starts by writing a numbered plan with an acceptance check per task; each task becomes one small, described change; the checks must be green before every change is saved.
- **Decisions are frozen.** An assistant that finds a decision wrong does not quietly work around it; it writes a one-page proposal and waits. This is how the six new decision records came about.
- **Assumptions are checked, not believed.** We had access to a real shared cluster and cloud; every claim about what such an environment allows was tested with a command and the output kept.

8. Where we are now, honestly

The design, the rules and the automatic checks exist and are proven by tests. The data pipeline itself is the next block of work. We prefer to say this plainly rather than let a green checkmark suggest more than it means.

Piece	Status	What exists today
Data scope and source contracts	Done	Which sources, exact operations, units, time rules, licences, what is deliberately excluded.
Assumptions, verified against a real environment	Done	Shared cluster rights, cloud quotas, storage features, identity, internet access: each tested, output recorded.
Architecture decisions	Done	27 written decision records, including the six that came out of the independent review.
Repository skeleton with enforced rules	Done	Code style, strict typing, layer rules, no-network tests, folder and import limits for new sources, dependency allow-list, secret scan, lock consistency, pinned tooling. 36 tests, several of which exist only to prove a gate rejects a bad change.
Independent review answered	Done	All 14 findings accepted and closed by a decision record, a gate with a test, or a dated task. Two need the author: a real code host, and the original research evidence files.
Contracts in code (recipe format, clean-table format)	Next	Planned in detail; example recipes for six kinds of source.
Core library: fetch, archive, process, store, run log	Next	Planned module by module, with the failure drills of Diagram 2 as tests.
Newcomer tooling and the full gate list	Next	Scaffold command, sample recorder, "needs approval" result, one bad example per rule.
The four sources, added through that tooling	Next	Three by us, the fourth strictly as a newcomer would.
Deployment code, backups, drills, dashboards	Next	Laptop, shared cluster, own cluster; restore and rollback rehearsals.
Blind test and final packaging	Next	Fresh AI session and a junior, given only the README.

What the independent review changed

Before writing the pipeline, we handed the repository to a second AI system with the instruction to judge it against the original brief and to try to get bad changes past the checks. It succeeded four times: a hidden download slipped past the lint rules, a failing test in a new source folder was simply

never run, a password-like string next to the word "example" was ignored, and an out-of-date dependency list went unnoticed. It also found five places where our written rules contradicted each other. Every one of these is now either a test that fails on purpose or a decision record that settles the contradiction. Finding this before the code existed was cheap; finding it after would not have been.

Two things we need from the project owner

- A code hosting account and the maintainer's name, so that "only a human can merge" becomes an enforced setting rather than a written intention.
- The original folder of research evidence (saved website responses with checksums) that the data-scope document cites, or agreement that we regenerate it during the next phase.

9. Five words we could not avoid

Word	What we mean by it
Intraday market	The market where electricity for <i>today</i> is traded, right up to shortly before delivery, in 15-minute blocks. Prices change block by block. OTE runs it for the Czech Republic.
Recipe (in the code: "manifest")	A short text file describing one data source: where to fetch, how often, the licence, which measurements it contains and in what units. The platform reads it; nobody writes fetching code.
Archive (in the code: "Bronze")	The store of original downloaded files, kept unchanged and in two places. The clean table (in the code: "Silver") can always be rebuilt from it.
Pull request	A proposed change to the code that others can see, that automatic checks run on, and that a person approves before it takes effect.
Decision record	A one-page note saying what we decided, why, what we rejected and what it costs if we are wrong. Frozen once accepted; changed only by a new record. There are 27.

Where to look in the repository

You want to know...	Read
What data and why	docs/01-data-scope.md
What we assumed and what we verified	docs/00-assumptions.md
The design and the decisions	docs/02-architecture-decisions.md, docs/adr/
The plan, phase by phase	docs/03-roadmap.md, docs/progress.md
The independent review and our answer	codex-review/, docs/reviews/

The rules an assistant must follow	CLAUDE.md (identical to AGENTS.md)
The tests that prove the gates work	tests/harness/

Generated 19 September 2026 from the repository at that date. Diagrams are simplified on purpose; the decision records carry the exact mechanisms.